

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Computer Vision with OpenCV

COURSE CODE: DSA0202

COURSE OUTCOME COVERED

CO5: Apply computer vision techniques to real-world applications such as face recognition, tracking, medical imaging, and autonomous systems. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Total Marks:50

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I P Sai Bhavyatha(192424333)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Palacharla Sai Bhavyatha

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	

Constraint-Based Problem Solving

1. Scenario: A company wants to deploy a secure access system using face recognition in a low-light environment.

Constraints:

- A. Limited computational resources (embedded device)
- B. Faces may appear at different orientations ($\pm 30^\circ$)
- C. Real-time recognition required (< 200 ms per face)

Questions:

- 1. How would you adapt Eigenface-based recognition to handle varying illumination and orientation?
- 2. Suggest preprocessing steps to improve recognition under low-light conditions.
- 3. Discuss strategies to optimize computation for real-time performance on constrained hardware.

Question 2: Photo Album Organization

2. Scenario: You are designing an AI system to automatically organize a large personal photo collection by people and events.

Constraints:

- A. Millions of images with inconsistent resolution and quality
- B. Multiple photos per person in different poses
- C. Privacy considerations: all processing must occur locally

Questions:

- 1. How would you use face recognition and clustering to group photos of the same individual?
- 2. Propose a method to detect and separate images with occlusions or partial faces.
- 3. Discuss trade-offs between accuracy and computational efficiency for local processing.

3. Scenario: A smart city surveillance system must detect moving objects and track pedestrians across multiple cameras.

Constraints:

- A. Crowded scenes with frequent occlusion
- B. Limited network bandwidth between cameras
- C. Real-time processing requirement

Questions:

1. How would you design a foreground–background separation algorithm to minimize false positives in crowded scenes?
2. Explain how particle filters can be used to handle occlusion during tracking.
3. Propose an efficient multi-camera strategy for consistent pedestrian identification across viewpoints.

4. Scenario: A self-driving car must navigate urban streets while detecting lanes, road signs, and pedestrians.

Constraints:

- A. Variable weather and lighting conditions (rain, night)
- B. Real-time decision-making (≤ 50 ms per frame)
- C. High-density traffic with moving pedestrians

Questions:

1. How would you implement road detection robust to shadows, rain, and nighttime illumination?
2. Propose a pipeline to simultaneously detect road signs and pedestrians efficiently.
3. Discuss the trade-offs between deep learning-based detectors and classical computer vision methods under real-time constraints.

5. Scenario: An AR system for surgical assistance must overlay segmented organs in real time.

Constraints:

- A. High-resolution 3D medical images (CT/MRI)
- B. Latency must be < 100 ms for surgeon interaction
- C. Limited GPU memory on wearable AR headset

Questions:

1. How would you implement real-time organ segmentation under these memory and latency constraints?
2. Discuss methods to integrate segmentation results accurately in AR/MR overlays.
3. Suggest strategies to reduce computational load while maintaining segmentation accuracy.

ANSWERS

Question 1: Secure Face Recognition in Low-Light Environment

1. Adapting Eigenface-Based Recognition for Varying Illumination and Orientation

Eigenface-based recognition uses Principal Component Analysis (PCA) to represent faces using a smaller set of important features. However, Eigenfaces are sensitive to changes in lighting and face orientation. Therefore, the system should be adapted using proper preprocessing and multiple training representations.

First, the input face should be detected and aligned using facial landmarks such as the eyes, nose, and mouth. Since faces may appear within $\pm 30^\circ$ orientation, the system can use face alignment and image warping to normalize tilted faces. Multiple Eigenface models can also be trained using frontal and slightly rotated faces.

To handle illumination changes, histogram equalization or CLAHE can be applied before PCA. Illumination normalization methods such as gamma correction and Difference of Gaussian filtering can further reduce lighting variations.

The complete process can be:

Face Detection → Alignment → Illumination Normalization → PCA/Eigenface Feature Extraction → Face Matching → Identity

This approach improves recognition accuracy while keeping the Eigenface method computationally lightweight for an embedded device.

2. Preprocessing Steps for Low-Light Conditions

The following preprocessing steps can improve recognition under low-light conditions:

Noise Reduction: Apply a Gaussian or median filter to reduce sensor noise.

Brightness Enhancement: Use gamma correction to improve dark facial regions.

Contrast Enhancement: Apply histogram equalization or CLAHE to improve local contrast.

Illumination Normalization: Reduce the effect of uneven lighting using normalization techniques.

Face Detection: Detect the face and remove unnecessary background information.

Face Alignment: Align the eyes and other landmarks to obtain a consistent face position.

Image Resizing: Resize all faces to a fixed low resolution suitable for Eigenface processing.

Grayscale Conversion: Convert the image to grayscale because Eigenfaces mainly require intensity information.

These steps improve the quality and consistency of the input while reducing unnecessary computation.

3. Strategies to Optimize Computation for Real-Time Performance

Since the embedded device has limited computational resources and recognition must occur within 200 ms per face, computational efficiency is essential.

The system can use a small Eigenface feature space by retaining only the principal components that contain the most useful information. PCA significantly reduces the dimensionality of face images.

Other optimization strategies include:

- Use low-resolution aligned face images.
- Perform face detection only when necessary.
- Use lightweight face detection algorithms.
- Store precomputed Eigenface features for enrolled users.
- Compare the input feature vector using Euclidean or cosine distance.
- Use approximate nearest-neighbor search when the database is large.
- Avoid unnecessary image processing operations.
- Use integer or fixed-point operations where supported by the embedded hardware.

Therefore, the optimized pipeline can achieve real-time recognition while maintaining acceptable accuracy within the hardware limitations.

Conclusion:

The proposed solution satisfies all constraints by combining illumination normalization, face alignment, PCA-based dimensionality reduction, and lightweight matching techniques.

Question 2: Photo Album Organization

1. Using Face Recognition and Clustering to Group Photos

The system can automatically organize millions of photographs by combining face detection, face recognition, feature extraction, and clustering.

First, every image is processed to detect faces. Each detected face is aligned and converted into a compact face embedding using a lightweight face recognition model. Similar face embeddings are then grouped using clustering algorithms such as DBSCAN, K-Means, or hierarchical clustering.

For example:

Images → Face Detection → Face Alignment → Feature/Embedding Extraction → Similarity Calculation → Clustering → Person Groups

DBSCAN is particularly useful because it can identify groups without requiring the exact number of people in advance and can classify unusual images as noise.

For event organization, additional features such as date, location, image similarity, and scene information can be combined with the face clusters. This allows the system to create groups such as "Person A – Birthday Event" or "Person B – College Event."

Because processing must occur locally, the face embeddings and original photographs should remain on the user's device.

2. Detecting Images with Occlusions or Partial Faces

Occlusions occur when a face is partially covered by objects such as masks, sunglasses, hands, or other people.

The system can detect such cases using:

- Face detection confidence scores.
- Facial landmark detection.
- Percentage of visible facial landmarks.
- Face bounding-box size.
- Occlusion-aware quality estimation.
- Comparison between visible facial regions and stored face features.

If important landmarks are missing or the detected face is too small, the image can be marked as partial/occluded.

The system can then either exclude the image from automatic identity assignment or assign it using only reliable visible features. Multiple photographs of the same person can also help confirm identity when some images contain partial faces.

3. Trade-Off Between Accuracy and Computational Efficiency

For millions of images, processing every photograph with a large deep-learning model can require substantial computation and storage.

A practical solution is to use a two-stage approach:

Stage	1	—	Fast	Processing:
Use lightweight face detection, image resizing, duplicate detection, and basic quality filtering.				

Stage	2	—	Accurate	Processing:
Apply a stronger recognition model only to images containing high-quality or ambiguous faces.				

Accuracy increases when larger models and higher-resolution images are used, but computational cost also increases. Smaller models and lower resolutions provide faster processing but may reduce recognition accuracy.

Since privacy requires local processing, the system should prefer lightweight models, batch processing, CPU/GPU acceleration where available, and local encrypted storage.

Conclusion:

A hybrid recognition and clustering system provides a good balance between privacy, accuracy, scalability, and computational efficiency.

Question 3: Smart City Surveillance System

1. Foreground–Background Separation in Crowded Scenes

Foreground-background separation identifies moving objects by separating them from the background. In crowded environments, a simple background subtraction method can produce many false positives because of shadows, lighting changes, and overlapping pedestrians.

A robust approach is to use an adaptive background model such as Mixture of Gaussians

(MOG/MOG2) or another adaptive background subtraction technique.

The process can be:

Video Frame → Background Modeling → Foreground Extraction → Shadow Removal → Morphological Filtering → Object Detection

To minimize false positives:

- Continuously update the background model.
- Use shadow detection.
- Apply morphological opening and closing.
- Remove very small regions.
- Use object-size and shape constraints.
- Combine foreground masks with a pedestrian detector.
- Track objects across multiple frames rather than relying on a single frame.
- This produces a more reliable foreground mask even in crowded scenes.

2. Using Particle Filters to Handle Occlusion

A particle filter is a probabilistic tracking technique that represents the possible position of an object using multiple particles.

During normal tracking, particles are concentrated around the predicted position of the pedestrian. When the pedestrian becomes partially or completely occluded, the visual observation becomes unreliable.

The particle filter can continue predicting the person's location using the previous position and motion information. When the pedestrian becomes visible again, the particles are updated using the new observation.

The basic process is:

Prediction → Observation → Weighting → Resampling → Position Estimation

This makes particle filters useful for handling temporary occlusions because the tracker does not immediately lose the pedestrian when visual information becomes unavailable.

3. Efficient Multi-Camera Pedestrian Identification

A multi-camera system can use a combination of person detection, tracking, and re-identification (Re-ID).

Each camera performs local processing instead of continuously sending raw video to a central server. Because network bandwidth is limited, cameras should transmit only compact information such as:

- Object ID
- Timestamp
- Bounding box
- Motion information
- Compact appearance/Re-ID feature vector

When a pedestrian moves between cameras, the Re-ID feature can be compared with stored features to determine whether the same person has appeared in another camera.

The architecture can be:

Camera → Local Detection → Local Tracking → Feature Extraction → Compact Metadata → Central Re-ID/Tracking

This reduces network traffic while maintaining consistent identification across camera viewpoints.

Conclusion:

Combining adaptive background subtraction, particle-filter tracking, and compact Re-ID features provides an efficient solution for crowded multi-camera surveillance.

Question 4: Self-Driving Car

1. Road Detection Under Shadows, Rain, and Nighttime Conditions

Road detection must remain reliable under different illumination and weather conditions. A robust system should combine image preprocessing with lane and road segmentation.

For daytime images, color-space conversion such as HSV or HLS can help distinguish road surfaces from other regions. Edge detection and perspective transformation can then be used to identify lane boundaries.

For shadows and changing illumination:

- Apply contrast normalization.
- Use adaptive thresholding.
- Apply illumination correction.
- Use gradient-based features.
- Combine color and edge information.

For nighttime conditions, image enhancement and low-light preprocessing can improve visibility.

Under rain, temporal information from consecutive frames can help distinguish road features from temporary reflections.

A robust pipeline is:

Camera Input → Image Enhancement → Road/Lane Segmentation → Shadow/Rain Handling → Lane Detection → Road Boundary Estimation

Combining multiple visual features makes the system more reliable than using a single threshold.

2. Pipeline for Detecting Road Signs and Pedestrians

The system should use a shared preprocessing stage followed by specialized lightweight detection modules.

Pipeline:

Camera Frame → Resize/Normalize → Object Detection → Road Sign Detection + Pedestrian Detection → Tracking → Decision Module

A lightweight object detector can identify pedestrians and road signs in a single frame. Tracking can then be applied across consecutive frames to reduce repeated detection and improve stability.

Region-of-interest processing can also be used to focus computation on important areas. For example, the upper and roadside regions can receive more attention for traffic signs, while road-level regions can be prioritized for pedestrians and vehicles.

The final system combines detection results with vehicle position and motion information before making driving decisions.

3. Deep Learning vs Classical Computer Vision

Factor	Deep Learning	Classical Computer Vision
Accuracy	Generally higher in complex environments	Lower in highly variable conditions
Feature extraction	Learns features automatically	Requires manually designed features
Weather robustness	Better when trained on diverse data	More sensitive to lighting and weather
Computational cost	Usually higher	Usually lower
Hardware requirement	Often needs GPU/NPU acceleration	Can run efficiently on CPU
Training requirement	Requires large datasets	Requires less training data
Adaptability	High after suitable training	Limited
Real-time performance	Requires optimization	Usually faster

For the ≤ 50 ms per frame constraint, a lightweight deep-learning detector with quantization and hardware acceleration can provide a good balance between accuracy and speed.

Classical computer vision can be used for simpler tasks such as lane detection, while deep learning can handle complex object detection.

Conclusion:

A hybrid architecture combining classical computer vision and optimized deep learning provides an effective balance between accuracy and real-time performance.

1. Real-Time Organ Segmentation Under Memory and Latency Constraints

The system must process high-resolution CT/MRI data while maintaining latency below 100 ms and operating with limited GPU memory.

A lightweight segmentation network should therefore be selected instead of a very large model. The input can be divided into smaller 3D patches or regions of interest (ROI) so that the complete high-resolution volume does not have to be processed simultaneously.

The pipeline can be:

CT/MRI Input → Preprocessing → ROI/Patch Extraction → Lightweight Segmentation Network → Post-processing → 3D Organ Model → AR Overlay

Computational efficiency can be improved using:

- Smaller input resolutions where clinically appropriate.
- Patch-based processing.
- Lightweight CNN architectures.
- Model quantization.
- Reduced precision computation.
- GPU memory management.
- ROI-based segmentation.
- Reusing previous frame information.

Only the required anatomical region should be processed at high detail.

2. Integrating Segmentation Results into AR/MR Overlays

The segmentation output must be accurately aligned with the patient's real-world anatomy.

First, the segmented organ is converted into a suitable 3D representation. The system then performs registration between the medical image coordinate system and the real-world AR coordinate system.

Important techniques include:

- Image-to-patient registration.
- Camera and headset calibration.
- Spatial tracking.
- Coordinate transformation.
- 3D surface reconstruction.
- Continuous alignment verification.

The overlay should be anchored to stable anatomical reference points. Tracking information from the AR headset can continuously update the position of the virtual organ.

The process is:

Medical Image → Segmentation → 3D Reconstruction → Registration → Spatial Tracking →

AR/MR Overlay

Accurate registration is essential because even a small alignment error can make the overlay misleading.

3. Reducing Computational Load While Maintaining Accuracy

Several optimization techniques can reduce computational requirements:

- Model Compression: Remove unnecessary parameters from the network.
- Quantization: Use lower numerical precision to reduce memory and computation.
- Knowledge Distillation: Train a smaller model using knowledge from a larger model.
- Patch-Based Processing: Process only relevant sections instead of the complete volume.
- ROI Processing: Focus computation on the anatomical region of interest.
- Hardware Acceleration: Use available GPU/NPU acceleration efficiently.
- Frame-to-Frame Reuse: Reuse previous segmentation results when the scene changes only slightly.
- Efficient Network Architecture: Select a lightweight segmentation model.
- Multi-resolution Processing: Use lower resolution for initial detection and higher resolution only where required.

These methods reduce memory usage and processing time while preserving important segmentation accuracy.

Conclusion:

A lightweight, ROI-based, compressed segmentation pipeline combined with accurate registration can satisfy the <100 ms latency and limited GPU-memory constraints while providing reliable real-time AR visualization.

Overall Conclusion

All five scenarios can be solved effectively by following a constraint-based problem-solving approach. First, the computational, environmental, privacy, latency, and accuracy constraints are identified. Next, appropriate computer vision, machine learning, preprocessing, optimization, and tracking techniques are selected. Finally, the proposed solutions are optimized to satisfy the specific constraints.

The solutions demonstrate the application of concepts such as Eigenfaces, PCA, clustering, background subtraction, particle filters, object detection, deep learning, image segmentation, registration, and model optimization. Each solution balances accuracy, computational efficiency, real-time performance, and practical constraints, thereby providing feasible and logically justified solutions.